

Conditional Autoregressive Diffusion for Procedural Content Generation

Ahmed E. Al Mohamady*, Shehab T. Shaban*, Mohamed A. Ebrahim*, Ahmed B. Zaky*[†]

*Egypt-Japan University of Science and Technology (E-JUST), Alexandria, 5221241, Egypt

[†]Department of Electrical Engineering, Benha University, Banha, 13511, Qalubiya, Egypt

Emails: {ahmed.elmohamady, shehab.tarek, mohamed.aly}@ejust.edu.eg, ahmed.zaky@feng.bu.edu.eg

Abstract—Diffusion models have been the state-of-the-art models for continuous content generation, such as image or audio generation, for a long time, but their use in discrete problems remains underexplored. To address this gap, we decided to use one of the most famous and prominent games in history, Super Mario Bros., which is a 2D platformer game with a discrete number of tiles that represent a game level. Game level generation has always been a difficult problem for deep learning methods, as they require extensive training data, which is usually not available for video games. Our method aims to show the potential of diffusion models in discrete problems such as game level generation using a very limited training dataset.

Index Terms—Procedural content generation, diffusion models, level generation, deep learning

I. INTRODUCTION

Procedural Content Generation (PCG) has been a cornerstone of game development for a long time, enabling infinite replayability and reducing manual design costs. Super Mario Bros., one of the most iconic games ever made, presents unique challenges for PCG due to the intricate balance between challenge and aesthetics.

Traditional PCG methods for platformer games rely on rule-based systems, grammars, or evolutionary algorithms [1]. While effective, these approaches require extensive domain expertise and struggle to capture the more complex and subtle design patterns present in human-created levels. Recent advances in deep learning, particularly generative models, offer promising alternatives that can learn implicit design rules directly from data.

Diffusion models have been a popular choice for generative models in recent years, demonstrating impressive results in image synthesis, audio generation, and molecular design [2]. However, their application to structured discrete domains remains mostly unexplored. This work addresses this gap by introducing a conditional autoregressive latent diffusion framework specifically designed for 2D platformer level generation.

A. Contributions

Our main contributions through this paper are as follows:

- We propose a novel autoregressive architecture that has not been explored in this field before according to our research.
- We show that diffusion models, originally designed for continuous domains, can be successfully adapted to discrete tile-based PCG.

- We introduce a difficulty-based conditional generation framework.
- We introduce a model that is much smaller than all currently available models while retaining comparable quality.
- Our method can generate levels of any size.

II. RELATED WORK

Many different methods have been attempted to automatically create game levels. This section is divided into two main groups exploring some of the most famous approaches: **traditional** and **machine learning methods**.

A. Traditional Methods

Older PCG systems mostly use set rules or optimizations.

Search-based PCG: These systems treat level generation as a search problem. They attempt to find the best level among many options using a fitness score (how good a level is). A general taxonomy and survey of search-based approaches was presented by Togelius et al. [3]. For example, researchers used **Monte Carlo Tree Search (MCTS)** to guide the creation of Super Mario Bros. levels based on Markov chain generation [4].

Evolutionary Algorithms (EAs): EAs use ideas from nature (such as survival of the fittest) to “evolve” a level. They start with many random levels and allow the best ones to survive and mix to create better levels over many cycles. EAs have been used to create Super Mario Bros. levels that balance fun and challenge [5]. Although these methods offer control, they are often limited in terms of the variety and creativity they can achieve. **Noise-based Procedural Generation:** Noise-based methods generate content by sampling functions. One of the most famous functions used for this is the **Perlin noise** function, as it was used for one of the most popular games to this day, **Minecraft**. These methods produce natural and smooth structures without explicitly defining rules for individual elements. While noise-based methods work well for high-level terrain synthesis, they struggle to capture complex patterns.

B. Machine Learning Methods

In recent years, deep learning models that can learn from data have transformed PCG [6].

Recurrent Neural Networks (RNNs): Models such as **LSTMs** (a type of RNN) were an early success. They treated a level as a long sequence of tiles (similar to a sentence) and learned the structural patterns. This showed that AI could learn the “grammar” of level design [7].

Generative Adversarial Networks (GANs) and Variational Autoencoders (VAEs):

- **VAEs** were used to create a flexible space where levels could be smoothly blended together or fixed if they were broken [8].
- **GANs** were used to allow the AI to design playable levels in a latent space [9]. The **TOAD-GAN** paper [10] is an important example. TOAD-GAN showed how to generate long, coherent levels in the style of a game (such as Super Mario Bros.) using only a single example level to train the model. This was a significant step toward style-based generation.

Diffusion Models: Diffusion models are powerful for generating data through a gradual **denoising** process (removing noise). They have had great success in image generation but have only recently been explored for structured data. One approach introduced using token embeddings instead of one-hot encoding along with the classic denoising network of diffusion models to generate coherent levels [11]. Another method used an unconditional UNet-based diffusion model to generate playable levels [12]. Another study demonstrates the ability to use diffusion for text-to-level generation by creating caption-level pairs; they used a text embedding model to automatically assign descriptive captions to an existing dataset [13]. So far, most recent research in this topic shows how diffusion is able to unconditionally generate discrete tiles, but their potential for conditional generation remains underexplored. Our method demonstrates the high potential for diffusion models in conditional generation of discrete problems.

III. METHODOLOGY

A. Problem Formulation

We represent Super Mario levels as 2D grids $L \in \mathbb{Z}^{H \times W}$ where each cell contains one of 13 tile types (ground, pipes, enemies, coins, etc.). Our goal is to learn a generative model $p(L)$ that produces diverse, playable levels that match the desired difficulty score given.

B. Game Elements

C. Level Patch Extraction

For each level in our dataset, a horizontal sliding window is used to extract overlapped patches with a stride of four.

D. Patch Difficulty Evaluation

Each level patch is represented as a 2D grid

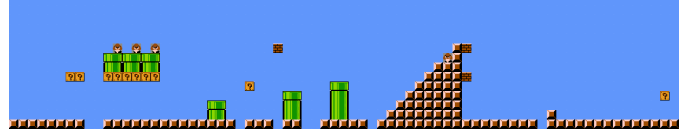
$$P \in \mathbb{Z}^{H \times W}, \quad (1)$$

where each cell encodes a discrete tile type. We define a continuous, normalized difficulty score

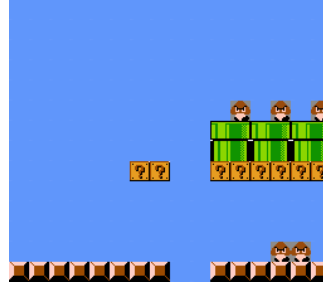
$$S(P) \in [0, 1] \quad (2)$$

TABLE I: Tile Symbol to Index Mapping

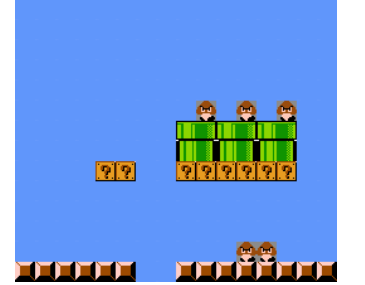
Tile Symbol	Index	Description
X	0	Solid ground
S	1	Solid breakable
-	2	Passable empty
?	3	Question block (full)
Q	4	Question block (empty)
E	5	Enemy
<	6	Top-left pipe
>	7	Top-right pipe
[	8	Left pipe
]	9	Right pipe
o	10	Coin
B	11	Cannon top
b	12	Cannon bottom



(a) Example Level



(b) Example patch 1



(c) Example patch 2

Fig. 1: Example patches from level

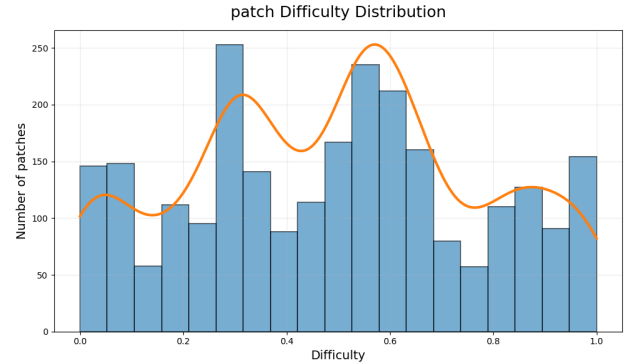


Fig. 2: Difficulty counts across all patches

that provides an approximation of local gameplay difficulty based on structural complexity.

1) **Feature Extraction:** From each patch P , five local difficulty indicators are extracted, each normalized by the patch width W .

Enemy Density:

$$d_e(P) = \frac{N_e(P)}{W}, \quad (3)$$

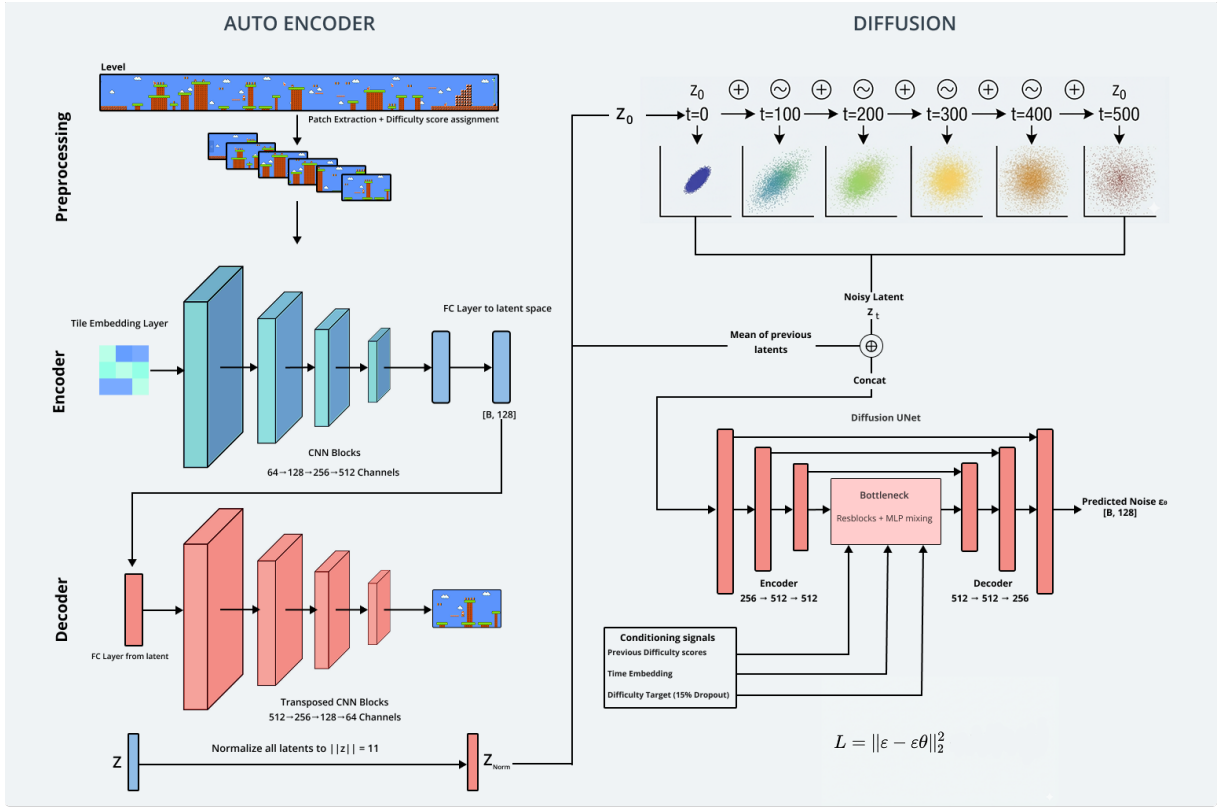


Fig. 3: High-level model architecture overview showing the diffusion/autoencoder pipeline.

where $N_e(P)$ denotes the total number of enemy tiles in patch P . **Canon Density:**

$$d_c(P) = \frac{N_c(P)}{W}, \quad (4)$$

where $N_c(P)$ represents the number of vertical columns containing cannon structures.

Pipe Density:

$$d_p(P) = \frac{N_p(P)}{W}, \quad (5)$$

where $N_p(P)$ denotes the number of distinct pipe columns.

Jump Density:

$$d_j(P) = \frac{N_j(P)}{W}, \quad (6)$$

where $N_j(P)$ is the number of ground-level gaps. Longer gaps are weighted more heavily:

$$N_j(P) = \sum_{g \in \text{gaps}} (1 + \max(0, (l_g - 2) \times 0.5)), \quad (7)$$

with l_g denoting the length of gap g in tiles.

Platform Density:

$$d_{pl}(P) = \frac{N_{pl}(P)}{W}, \quad (8)$$

where $N_{pl}(P)$ counts the number of elevated, floating platform segments. A platform is defined as a continuous horizontal sequence of solid tiles (Ground, Breakable, or Question blocks) above the ground floor with empty space beneath.

2) *Normalized Difficulty Score:* Each density term is normalized by a predefined maximum threshold and capped at 1.0:

$$\bar{d}_x(P) = \min\left(\frac{d_x(P)}{d_x^{\max}}, 1.0\right),$$

where $d_x^{\max}$ is set to 0.15 for enemies, cannons, and pipes, 0.20 for jumps, and 0.10 for platforms.

The final difficulty score is computed as a weighted linear combination of the normalized terms:

$$D(P) = \min\left(0.60 \bar{d}_e + 0.30 \bar{d}_c + 0.20 \bar{d}_j + 0.30 \bar{d}_{pl} + 0.20 \bar{d}_p, 1\right). \quad (9)$$

3) *Difficulty Assignment:* For a dataset of patches $\{P_i\}_{i=1}^N$, each patch is assigned a scalar difficulty value

$$S_i = D(P_i), \quad (10)$$

which is used directly as a conditioning signal for the diffusion model during training and sampling.

E. Architecture Overview

Our system consists of two main components as illustrated in Figure 3 as well as the generation process:

Stage 1: Autoencoder. We extract overlapping 14×16 patches from training levels with stride 4. The encoder processes each patch through tile embeddings (32-dim), four convolutional blocks with progressive channel expansion ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$), and a fully connected layer projecting to

a 128-dimensional latent space. The decoder mirrors this architecture with transposed convolutions. We train the conditional autoencoder using both the level patches and their difficulty score labels and pass them through the encoder; thus, the encoder learns to encode the label information into the latent space and the decoder learns to use the labels to reconstruct the original input. The objective is to minimize cross-entropy loss between the predicted logits and the ground-truth tiles.

Stage 2: Diffusion UNet. For each batch, noise is added to clean latents through a forward diffusion process over 500 timesteps using a linear noise schedule. The network is conditioned on three sources of information: (1) sinusoidal timestep embeddings, (2) autoregressive context derived from clean (noise-free) k previous latent patches $\{z_{i-k}, \dots, z_{i-1}\}$ and their respective difficulty scores, and (3) a target difficulty score embedded via Fourier features for smooth interpolation. During training, the target difficulty is dropped with 20% probability to enable classifier-free guidance. All signals are fused into a shared conditioning vector that additively modulates a deep residual MLP encoder–bottleneck–decoder architecture with skip connections.

The primary training objective is to minimize noise-prediction loss

$$\mathcal{L}_{\text{denoise}} = \mathbb{E}[\|\epsilon - \epsilon_\theta\|_2^2],$$

Stage 3: Generation. We start from pure noise, and at each reverse diffusion step the current noisy latent is passed through the diffusion model twice: once with the target difficulty score as conditioning, and once without (unconditional). The model outputs noise predictions for both cases. Classifier-free guidance interpolates between these predictions:

$$\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + s \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}}) \quad (11)$$

where s is the guidance scale controlling difficulty conditioning strength. This guided noise prediction is used in the reverse diffusion update. After generation, the latents are decoded through the autoencoder to tile patches, which are concatenated into a full level.

IV. EXPERIMENTS

A. Dataset

We train using only the Super Mario Bros. 1 and Super Mario Bros. 2 game levels from the VGLC dataset. The total number of levels used is 30, including 3 human-made levels, extracting 1298 total overlapping patches. **Preprocessing:** A few levels were removed from the dataset for their complexity, as well as relocating any floating enemies and fixing any mismatched tiles.

B. Training Details

Autoencoder: Trained for 100 epochs with Adam optimizer ($lr = 10^{-3}$, batch size 32), achieving 98.2% tile-level reconstruction accuracy.

Diffusion Model: Trained for 600 epochs with a linear learning rate schedule ($lr = 10^{-4} \rightarrow 10^{-6}$). We use 500 diffusion steps with a linear noise schedule. The model contains 10.4M parameters.

TABLE II: Difficulty Prediction Statistics per Target Difficulty

Target Difficulty	Generated Patch Difficulty	Error
0.00	0.000	0.000
0.20	0.125	0.075
0.40	0.479	0.079
0.60	0.250	0.350
0.80	0.521	0.279
1.00	1.000	0.0

TABLE III: Overall Difficulty Prediction Statistics

Metric	Value
Mean Absolute Error (MAE)	0.1305
Correlation Coefficient	0.8911
Total Samples Generated	6

C. Results

1) *Generation Quality:* Generated levels show strong structural consistency as shown in Figure 4. The autoregressive conditioning successfully maintains the level structure across patches, avoiding common artifacts like floating enemies or disconnected segments.

2) *Controllability:* By varying the difficulty condition $s \in \{0.0, 0.5, 1.0\}$, we observe corresponding changes in level difficulty as shown in Figure 4. Higher values produce more difficult patches containing jumps and enemies (note: stairs are not considered jumps in our difficulty evaluation metric).

3) *Accuracy:* Our model achieved a validation reconstruction loss of 0.1937. It was also able to generate levels that have very similar difficulty scores to the difficulty target given at sampling time, as shown in Table II.

V. DISCUSSION

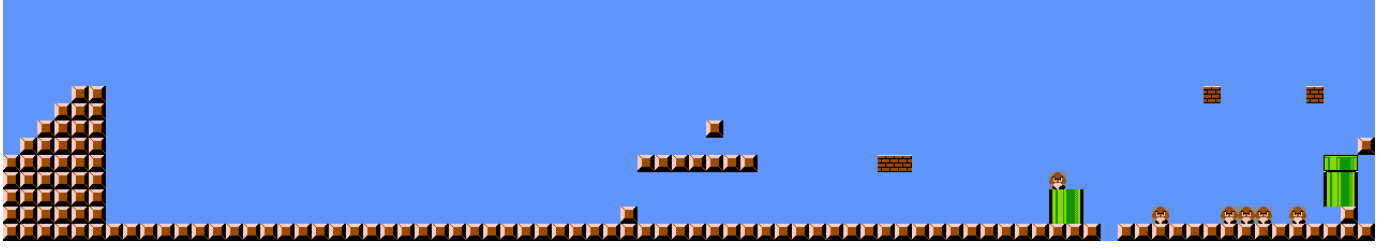
A. Advantages

Our approach offers several benefits over existing methods:

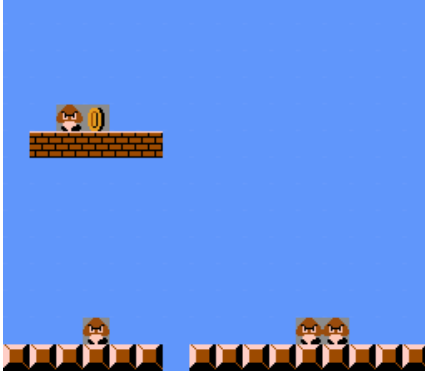
- **Controllability:** Explicit difficulty conditioning allows for more control over the generated levels.
- **Coherence:** Our autoregressive approach ensures structural consistency in the generated level.

VI. CONCLUSION

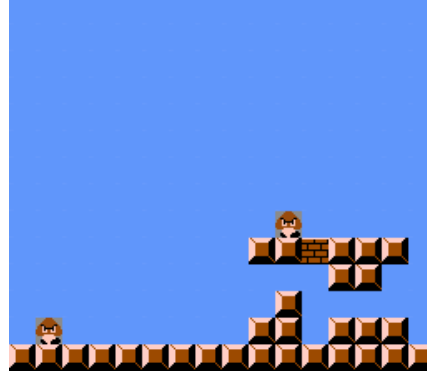
It was unexpected that a basic diffusion architecture made mainly for continuous data (images, audio, etc.) with slight modifications was able to work well for discrete problems like generating Super Mario levels. Honestly, the difficulty conditioning was the hardest part of this research, as the dataset used did not contain an even distribution of difficulty between the patches; most patches fell on the easier side, so it required some work to fix the data. Also, from the results shown in Table II, it is clear the model performs extremely well for target difficulty 0.0 and 1.0, but struggles slightly with target difficulties between 0.5 and 0.8. Future work could address the modifications made to the original dataset and allow for conditioning even with skewed difficulty scores, as well as improving the model’s performance for some difficulty values. Another addition would be to add element labels for each patch to represent what elements each patch has, which



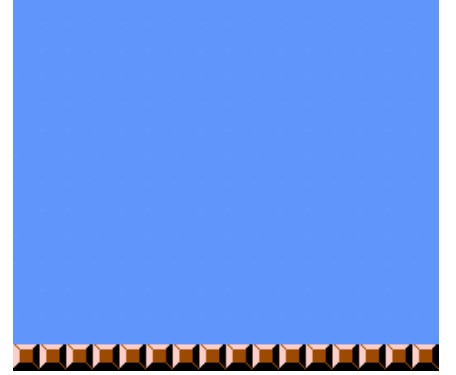
(a) Generated Level A. Target difficulty for this level is 1.0. When generating multiple patches per level, a difficulty schedule is applied to start from zero for the first patch and increase the difficulty with each new patch until reaching the target difficulty at the final generated patch. This figure clearly illustrates the clear increase of difficulty across the patches (later patches start having jumps and enemies).



(b) Generated Level B. Single patch with target difficulty of 1.0.



(c) Generated Level C. Single patch with target difficulty of 0.5.



(d) Generated Level D. Single patch with target difficulty of 0.0.

Fig. 4: Sample generated levels using the proposed autoregressive diffusion framework. The top image shows a full-width generated level, while the bottom row shows three additional examples with different difficulties.

would allow for conditional generation based on the desired elements in the level, not just difficulty.

REFERENCES

- [1] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Springer, 2016.
- [2] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [3] J. Togelius, G. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011.
- [4] A. Summerville and M. Mateas, “MCMCTS PCG: Mixed-initiative control for procedurally generated levels,” in *Proceedings of the FDG Workshop on Procedural Content Generation in Games*, 2015.
- [5] B. Horn, S. Dahlsgog, N. Shaker, and J. Togelius, “A comparative evaluation of procedural level generators in the Mario AI framework,” in *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2014.
- [6] A. Summerville, S. Snodgrass, M. Guzdial, C. Holmgård, A. Khalifa, N. S. Nielsen, J. Togelius, and G. N. Yannakakis, “Procedural content generation via machine learning (PCGML),” *IEEE Transactions on Games*, vol. 10, no. 3, pp. 257–270, 2018.
- [7] A. Summerville and M. Mateas, “Super Mario as a string: LSTM recurrent neural networks for level generation,” in *Proceedings of the DiGRA/FDG*, 2016.
- [8] A. Sarkar, S. Cooper, and J. Whitehead, “Blending levels from different games using variational autoencoders,” in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2020.
- [9] V. Volz, J. Schrum, J. Liu, S. Lucas, A. Smith, and S. Risi, “Evolving Mario levels in the latent space of a deep convolutional generative adversarial network,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2018.
- [10] M. Awiszus, F. Schubert, and B. Rosenhahn, “TOAD-GAN: Coherent style level generation from a single example,” *arXiv preprint arXiv:2008.01531*, 2020.
- [11] S. Dai, “Procedural level generation with diffusion models from a single example,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 9, pp. 10021–10029, 2024.
- [12] H. J. Lee, “Advancing level generation in Super Mario Bros.: A proposal for combining diffusion models with real-time playability assessment,” Ph.D. dissertation, Waseda University, Tokyo, Japan, 2024.
- [13] J. Schrum, B. Doe, C. Smith, and D. Lee, “Text-to-Level diffusion models with various text encoders for Super Mario Bros.,” *arXiv preprint arXiv:2507.00184*, 2025.